

Word2Vec

1. How do we represent the meaning of a word?

Definition: **meaning** (Webster dictionary)

- the idea that is represented by a word, phrase, etc.
- the idea that a person wants to express by using words, signs, etc.
- the idea that is expressed in a work of writing, art, etc.

Commonest linguistic way of thinking of meaning:

signifier (symbol) $\Leftrightarrow$ signified (idea or thing)

= denotational semantics

How do we have usable meaning in a computer?

Common solution: Use e.g. [WordNet](#), a thesaurus containing lists of **synonym sets** and **hypernyms** (“is a” relationships).

e.g. synonym sets containing “good”: e.g. hypernyms of “panda”:

```
from nltk.corpus import wordnet as wn
poses = { 'n': 'noun', 'v': 'verb', 's': 'adj (s)', 'a': 'adj', 'r': 'adv' }
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
        ", ".join([l.name() for l in synset.lemmas()])))
```

```
noun: good
noun: good, goodness noun: good, goodness
noun: commodity, trade_good, good adj: good
adj (sat): full, good adj: good
adj (sat): estimable, good, honorable, respectable adj (sat):
beneficial, good
adj (sat): good
adj (sat): good, just, upright
...
adverb: well, good
adverb: thoroughly, soundly, good
```

```
from nltk.corpus import wordnet as wn panda =
wn.synset("panda.n.01") hyper = lambda s:
s.hypernyms() list(panda.closure(hyper))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('object.n.01'),
Synset('physical_entity.n.01'),
Synset('entity.n.01')]
```

Problems with resources like WordNet

- Great as a resource but missing nuance
 - e.g. “proficient” is listed as a synonym for “good”. This is only correct in some contexts.
- Missing new meanings of words
 - e.g., wicked, badass, nifty, wizard, genius, ninja, bombest
 - Impossible to keep up-to-date!
- Subjective
- Requires human labor to create and adapt
- Can't compute accurate word similarity →

Representing words as discrete symbols

In traditional NLP, we regard words as discrete symbols:
hotel, conference, motel – a **localist** representation

Means one 1, the rest 0s



Words can be represented by **one-hot** vectors:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000)

Problem with words as discrete symbols

Example: in web search, if user searches for “Seattle motel”, we would like to match documents containing “Seattle hotel”.

But:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are **orthogonal**.

There is no natural notion of **similarity** for one-hot vectors!

Solution:

- Could try to rely on WordNet’s list of synonyms to get similarity?
 - But it is well-known to fail badly: incompleteness, etc.
- **Instead: learn to encode similarity in the vectors themselves**

Recall: Creating Numerical Features from Text

Code

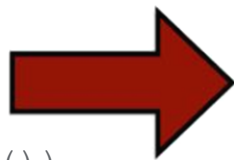
```
import pandas as pd
from sklearn.feature_extraction.text
    import CountVectorizer

corpus = ['This is the first document.',
          'This is the second
document.',
          'And the third one.']

cv = CountVectorizer()
X = cv.fit_transform(corpus)
pd.DataFrame(X.toarray(),
             columns=cv.get_feature_names())
```

Output

	and	document	first	is	one	second	the	third	this
0	0	1	1	1	0	0	1	0	1
1	0	1	0	1	0	1	1	0	1
2	1	0	0	0	1	0	1	1	0



Count Vectorizer

Word/Document Vectors with CountVectorizer

Document Vectors

- Doc 0: [0, 1, 1, 1, 0, 0, 1, 0, 1]
- Doc 1: [0, 1, 0, 1, 0, 1, 1, 0, 1]
- Doc 2: [1, 0, 0, 0, 1, 0, 1, 1, 0]

	and	document	first	is	one	second	the	third	this
0	0	1	1	1	0	0	1	0	1
1	0	1	0	1	0	1	1	0	1
2	1	0	0	0	1	0	1	1	0

Flip it around → Word Vectors

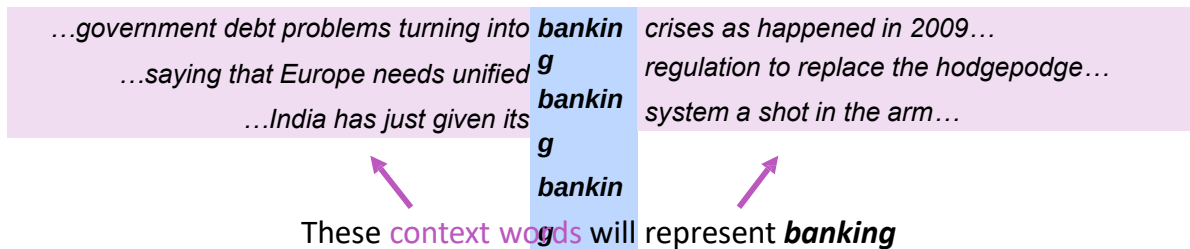
- and: [0, 0, 1]
- document: [1, 1, 0]
- first: [1, 0, 0]
- is: [1, 1, 0]
- one: [0, 0, 1]
- second: [0, 1, 0]
- the: [1, 1, 1]
- third: [0, 0, 1]
- this: [1, 1, 0]

	0	1	2
and	0	0	1
document	1	1	0
first	1	0	0
is	1	1	0
one	0	0	1
second	0	1	0
the	1	1	1
third	0	0	1
this	1	1	0

Representing words by their context



- Distributional semantics: **A word's meaning is given words that frequently appear close-by**
 - “You shall know a word by the company it keeps” (J. R. Firth 1957: 11)
 - One of the most successful ideas of modern statistical NLP!
- When a word w appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- Use the many contexts of w to build up a representation of w



Word vectors

We will build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.17 \\ 7 \\ -0.10 \\ 7 \\ 0.109 \\ -0.54 \\ 2 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Note: **word vectors** are sometimes called **word embeddings** or **word representations**. They are a **distributed** representation.

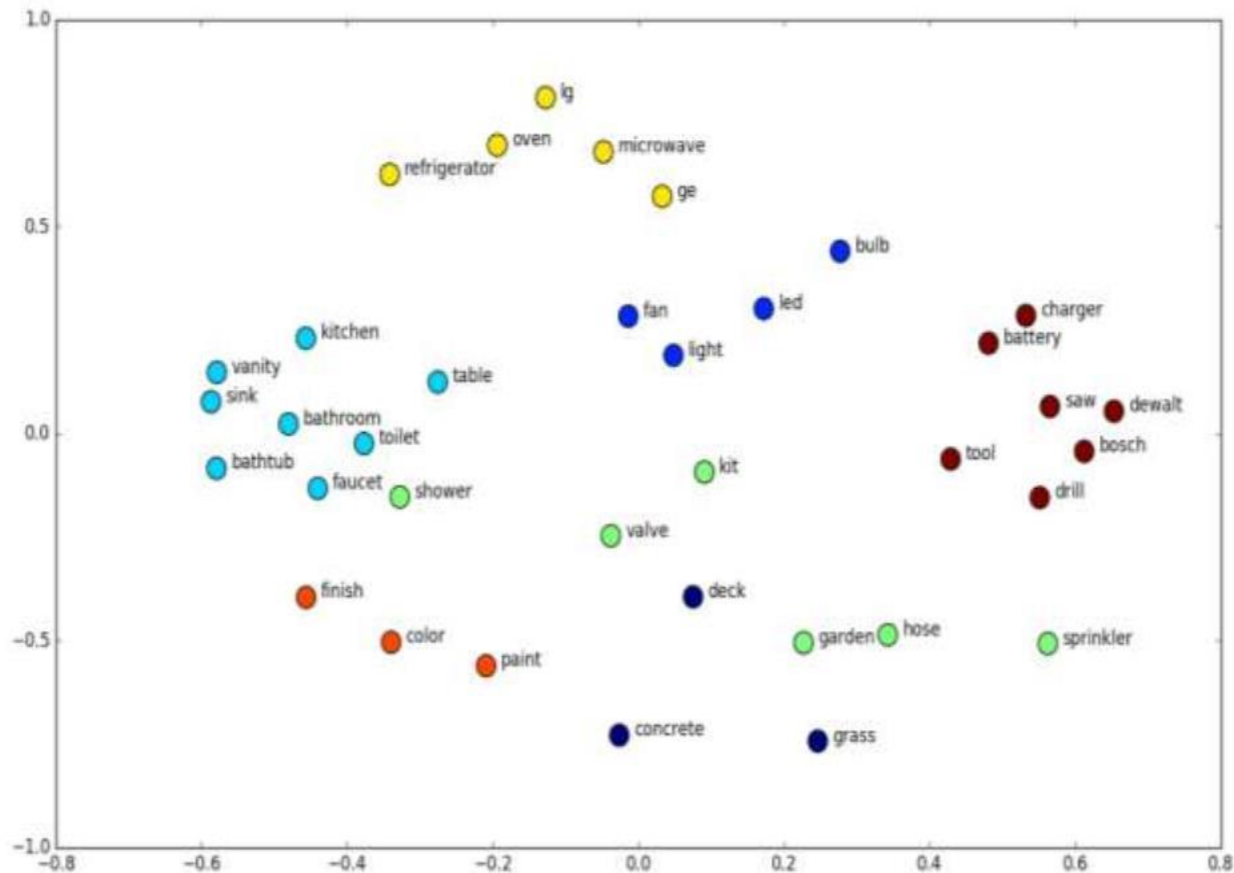
Word meaning as a neural word vector – visualization

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.17 \\ 7 \\ -0.10 \\ 7 \\ 0.109 \\ -0.54 \\ 2 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$


Why Word Vectors?

- Represent Conceptual “meaning” of words
- Word vectors close to each other have similar meaning



How to Use Word Vectors?

- Information Retrieval
- Document Vectors
 - A document vector is the average of its word vectors
- Machine Learning
 - Document Classification (from document vectors)
 - Document Clustering
- Recommendation
 - Recommend similar documents to search query or sample documents

Can we do better than counts?

- Answer: YES!
- Problems with counts:
 - Limited information
 - Possible Resolution: TFIDF
 - Vectors HUGE for many documents
 - Possible Resolution: Matrix Factorization
 - Bag of Words → No Word Order
 - Possible Resolution: Neural Networks → Word2vec!

Okay

Better

Best!

Binary/Counts

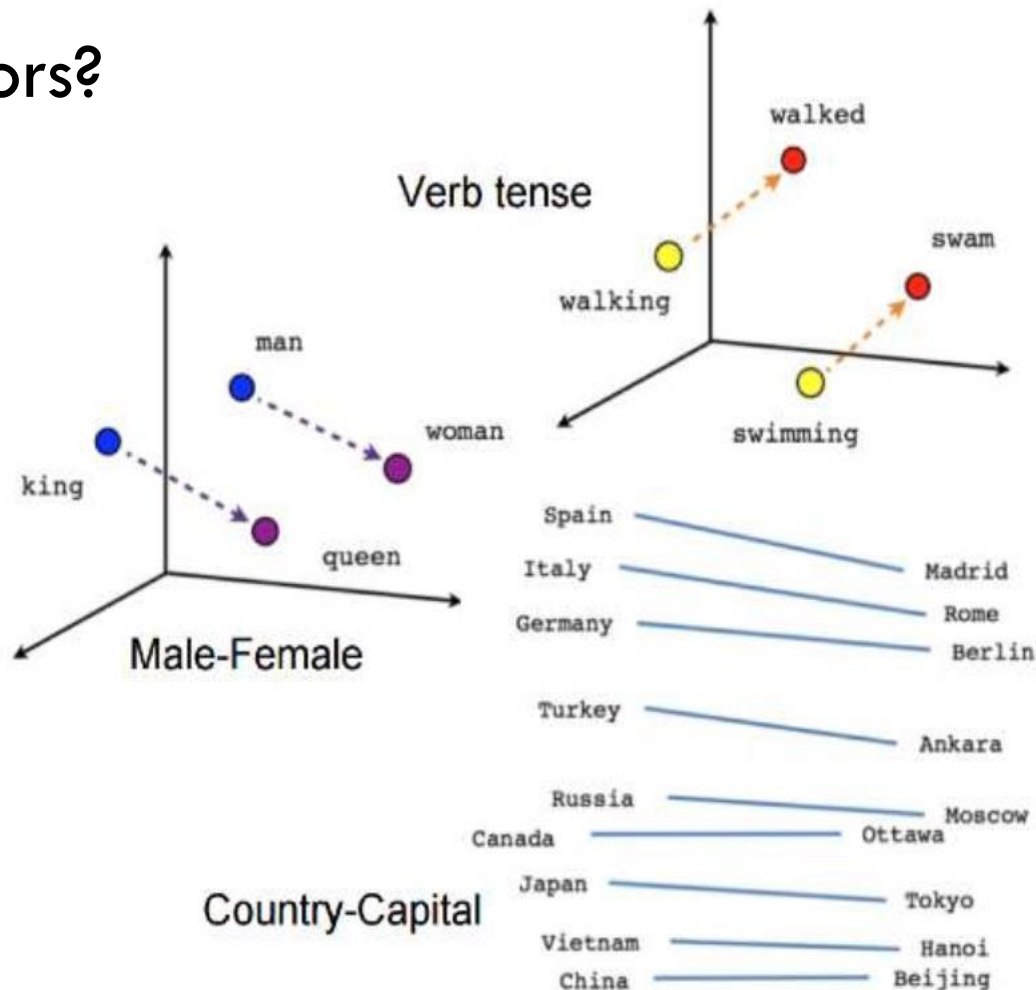
TFIDF

Dimensionality
Reduction

Word2Vec

How to Use Word Vectors?

- Answer: Comparability with human intuition
- Standard Baseline Tests:
 - Analogies
 - Ratings of Word Similarity
 - Verb Tenses
 - Country-Capital Relationships



Word2vec: Overview

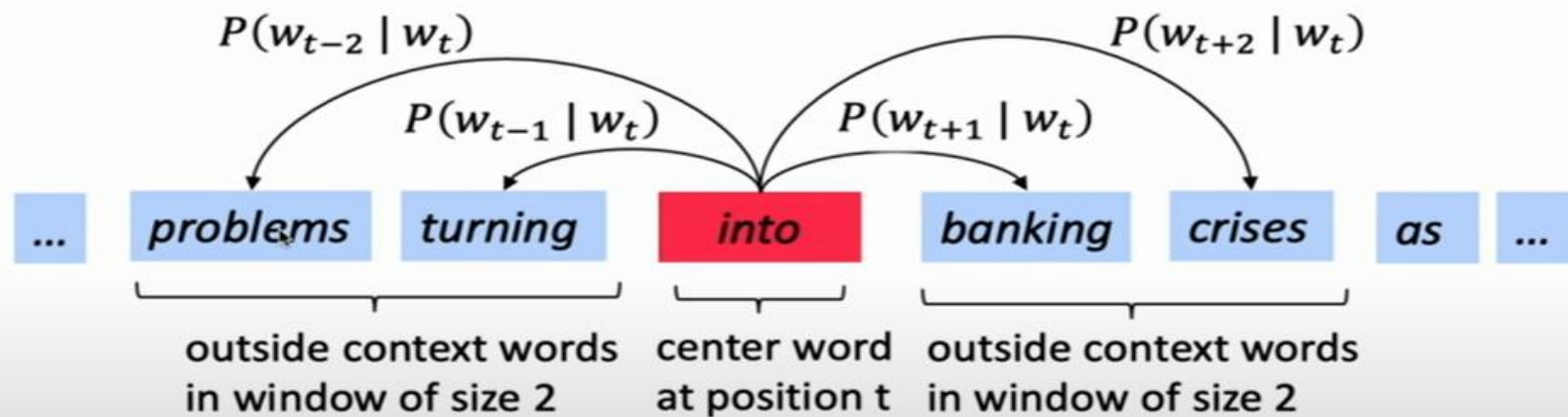
Word2vec (Mikolov et al. 2013) is a framework for learning word vectors

Idea:

- We have a large corpus of text
- Every word in a fixed vocabulary is represented by a vector
- Go through each position t in the text, which has a center word c and context (“outside”) words o
- Use the similarity of the word vectors for c and o to calculate the probability of o given c (or vice versa)
- Keep adjusting the word vectors to maximize this probability

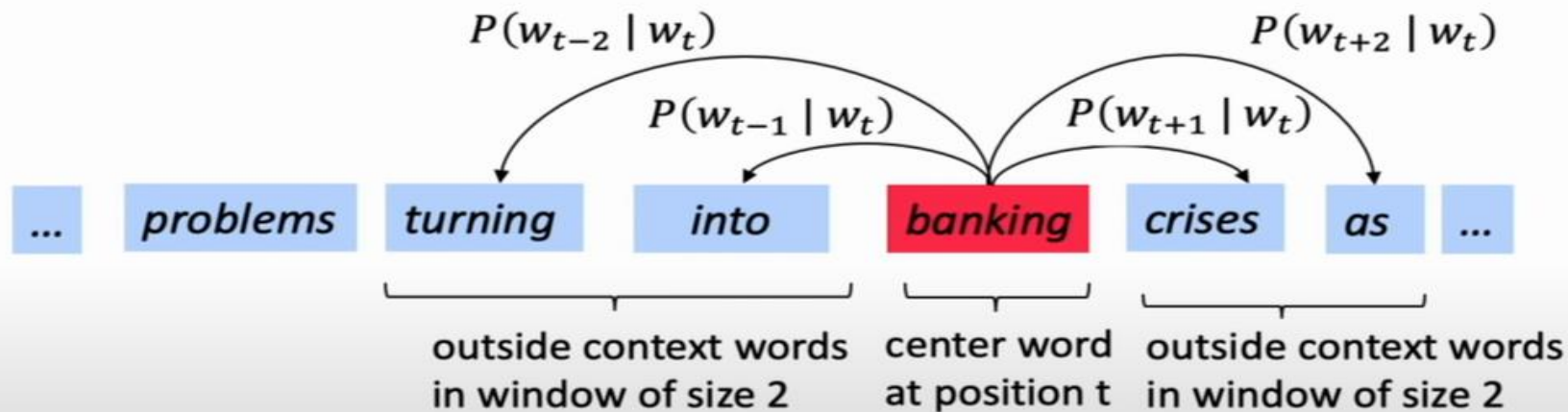
Word2Vec Overview

- Example windows and process for computing $P(w_{t+j} | w_t)$



Word2Vec Overview

- Example windows and process for computing $P(w_{t+j} | w_t)$



Word2vec: objective function

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_j .

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

θ is all variables
to be optimized

sometimes called *cost* or *loss* function

The **objective function** $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

Word2vec: objective function

- We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

- Question: How to calculate $P(w_{t+j} | w_t; \theta)$?
- Answer: We will use two vectors per word w :
 - v_w when w is a center word
 - u_w when w is a context word
- Then for a center word c and a context word o :

Word2vec: prediction function

Exponentiation makes anything positive

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Dot product compares similarity of o and c .

$$u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$$

Larger dot product = larger probability

Normalize over entire vocabulary to give probability distribution

- This is an example of the **softmax function** $\mathbb{R}^n \rightarrow \mathbb{R}^n$

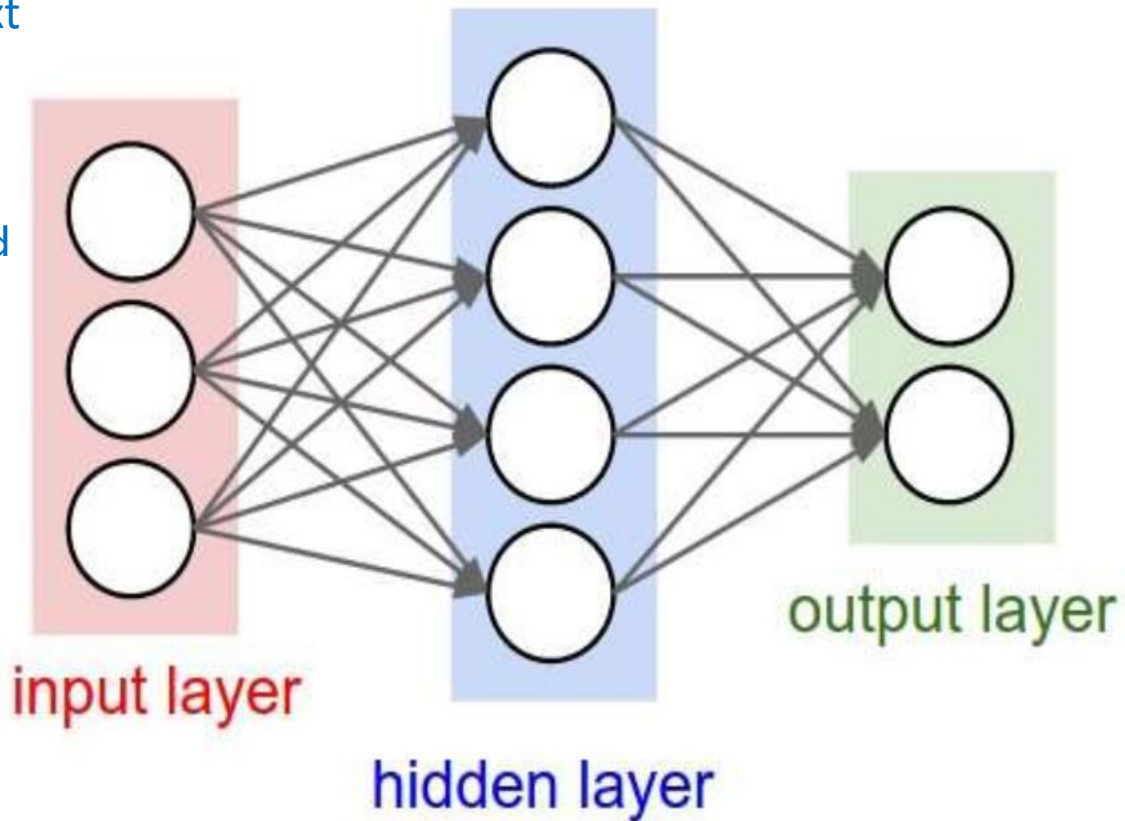
$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

- The softmax function maps arbitrary values x_i to a probability distribution p_i
 - “max” because amplifies probability of largest x_i
 - “soft” because still assigns some probability to smaller x_i
 - Frequently used in Deep Learning

Training Word2Vec

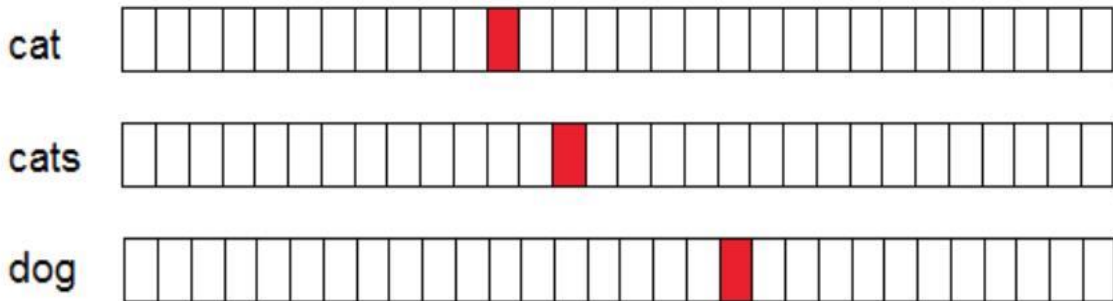
- Use a Neural Network on Context Windows
- 2 main approaches for inputs and labels:
 - Skip-Grams
 - Continuous Bag of Words (CBOW)

Vectors usually similar, subtle differences, also differences in computational time



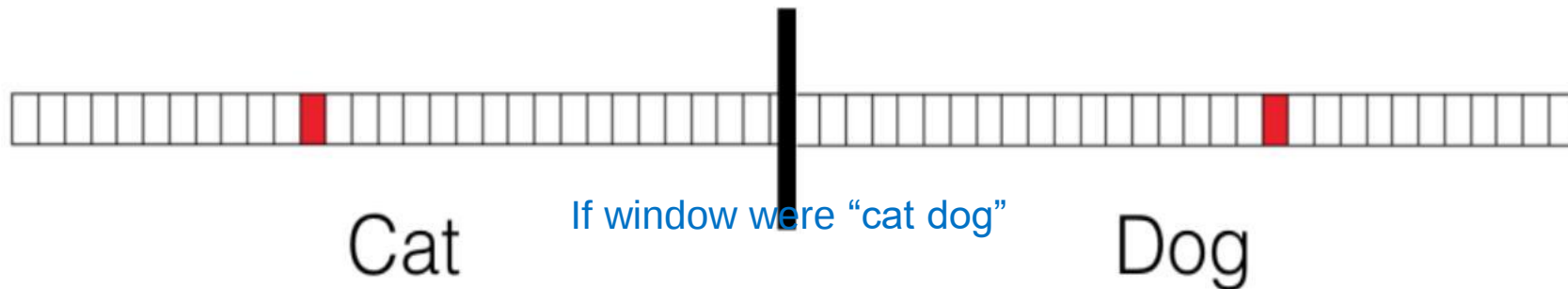
Training Word2Vec: One-Hot Word Vectors

- We need to be able to represent a sequence of words as a vector
- Assign each word an index from 0 to V
 - V is the size of the vocabulary aka # distinct words in the corpus
- A word vector is:
 - 1 for the index of that word
 - 0 for all other entries
- Called One-Hot Encoding



Training Word2Vec: One-Hot Context Windows

- Need vectors for context windows
- A window has vector that's the concatenation of its word vectors
- For window size d , the vector is of length $(V \times d)$
- Only d entries (one for each word) will be nonzero (1s)

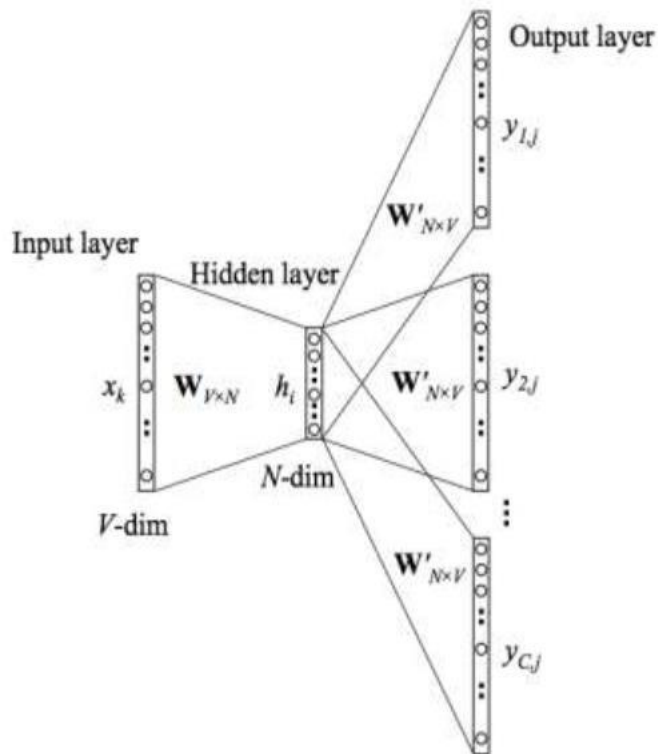


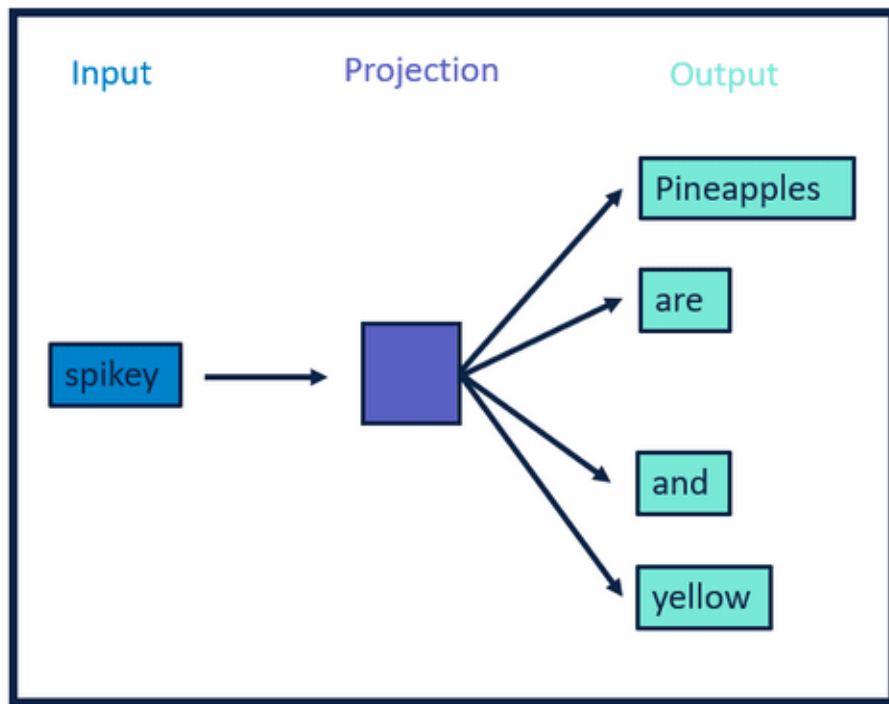
Training Word2Vec: SkipGrams

- SkipGrams is a neural network architecture that uses a word to predict the words in the surrounding context, defined by the window size.
- Inputs:
 - The middle word of the context window (one-hot encoded)
 - Dimensionality: V
- Outputs:
 - The other words of the context window (one-hot encoded)
 - Dimensionality: $(V \times (d-1))$

Training Word2Vec: SkipGrams

- SkipGrams architecture:





Skip-gram

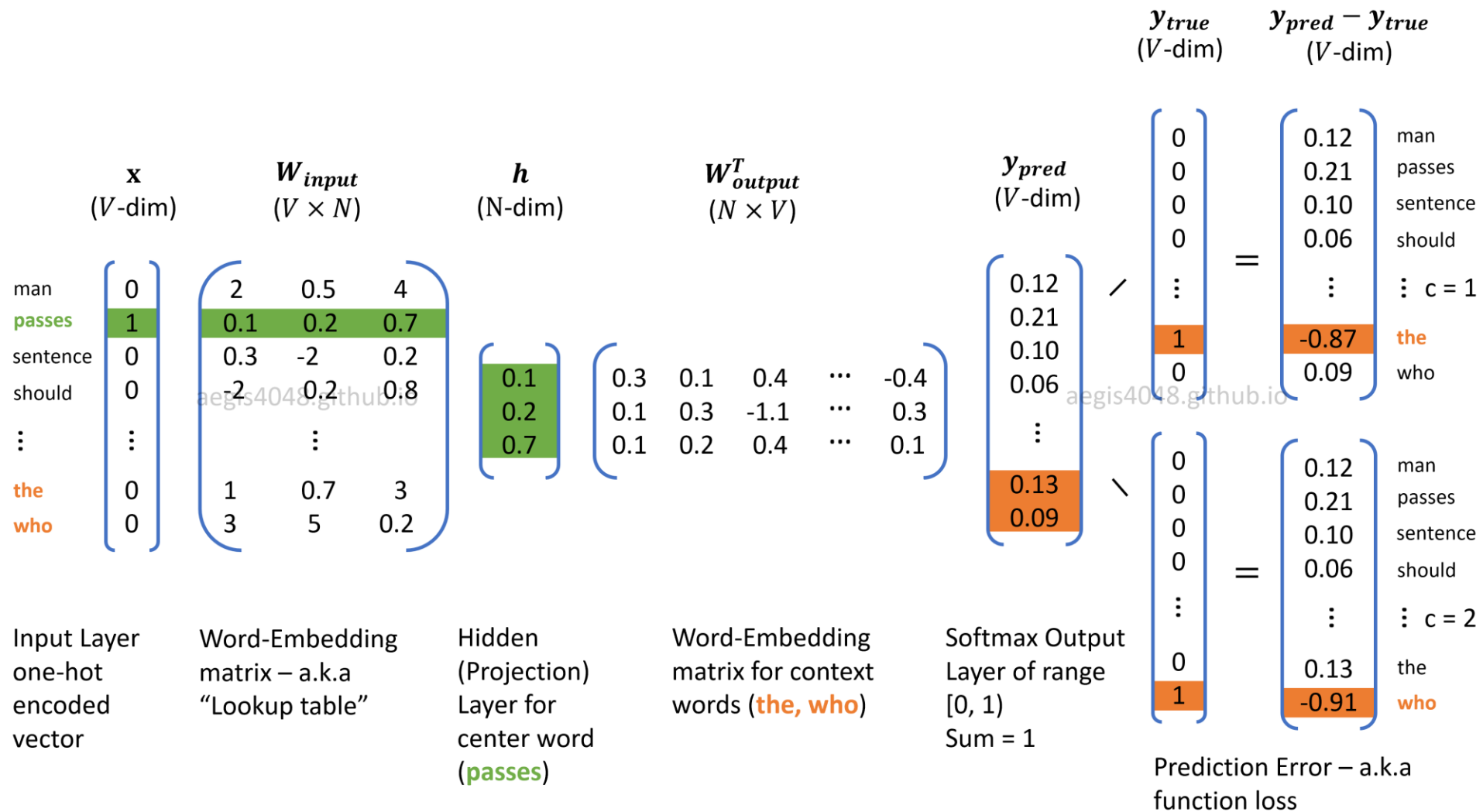
The man who passes the sentence
should swing the sword.

Source Text

The man who passes the sentence should swing the sword. →

Training
Samples

(passes, who)
(passes, the)

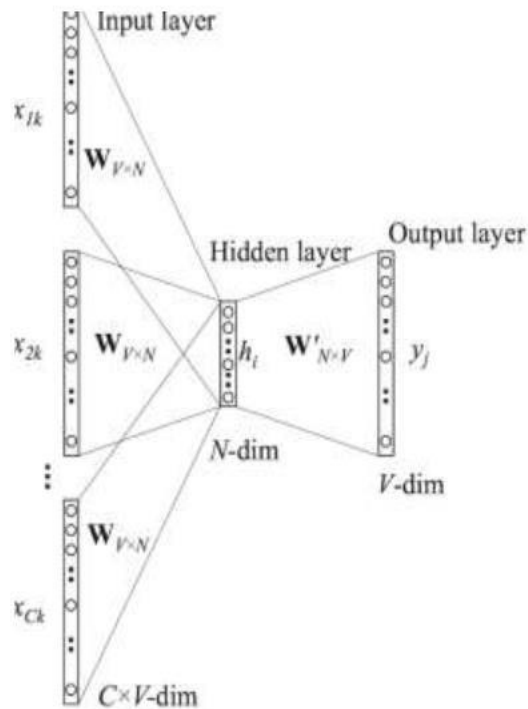


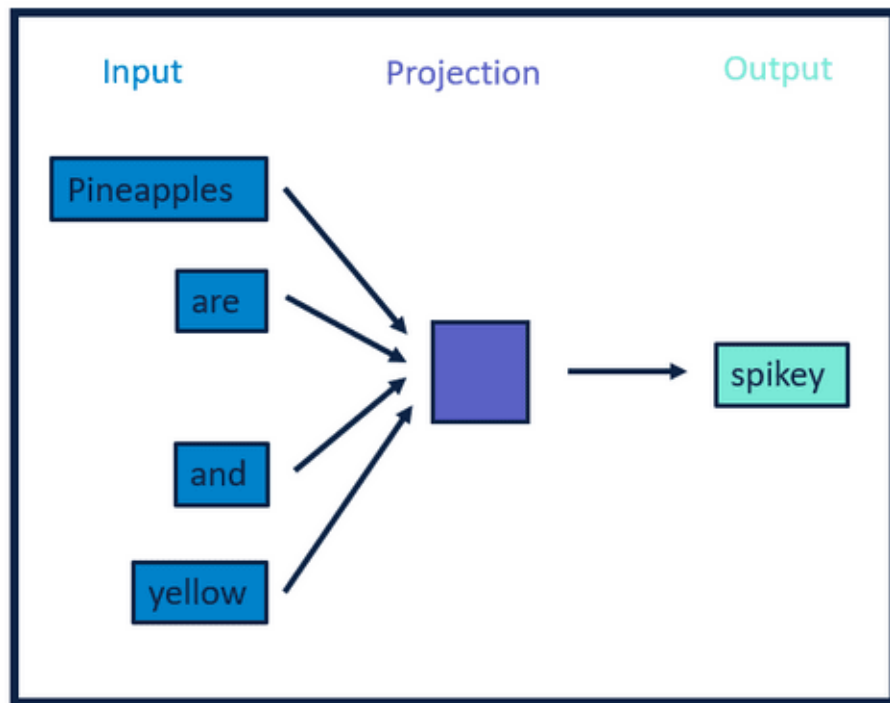
Training Word2Vec: CBOW

- CBOW (continuous bag of words) uses the surrounding context (defined by the window size) to predict the word.
- Inputs:
 - The other words of the context window (one-hot encoded)
 - Dimensionality: $(V \times (d-1))$
- Outputs:
 - The middle word of the context window (one-hot encoded)
 - Dimensionality: V

Training Word2Vec: CBOW

- CBOW architecture:





CBOW

Difference between CBOW and Skip-Gram

There are two models for Word2Vec: Continuous Bag Of Words (CBOW) and Skip-Gram. While Skip-Gram model predicts context words given a center word, CBOW model predicts a center word given context words. According to Mikolov:

Skip-gram: works well with small amount of the training data, represents well even rare words or phrases

CBOW: several times faster to train than the skip-gram, slightly better accuracy for the frequent words

Skip-Gram model is a better choice most of the time due to its ability to predict infrequent words, but this comes at the price of increased computational cost. If training time is a big concern, and you have large enough data to overcome the issue of predicting infrequent words, CBOW model may be a more viable choice. The details of CBOW model won't be covered in this post.